9/10/24

1. Genetic Optimization Algorithm

Define Fitness(individual)

population ← random individuals

Set population size, crossover rate, mutation rate and number of generations

Evaluate fitness of each individual

For each generation do

Select parents based on fitness

Apply crossover to selected parents to produce offspring

Apply mutation to offspring

Evaluate fitness of offspring

Create new population using offspring and best individuals

Update best solution

End FOR

Display best solution and its fitness.

Public Transport Scheduling

It is an optimization problem in which bus departure and arrival times must be planned efficiently to reduce passenger waiting time. If buses depart at inappropriate intervals, passengers may have to wait longer at bus stops, leading to inconvenience and overcrowding. The objective is to find an efficient bus schedule using a Genetic Algorithm.

The proposed solution generates different bus schedules and evaluates them based on passenger waiting time. Selection, crossover, and mutation are applied to improve the schedules while satisfying travel time and operating hour constraints. The algorithm outputs the best bus schedule with minimum passenger waiting time.

Algorithm: GA_Public_Transport_Scheduling

Define bus routes, passenger arrival times, travel times and scheduling constraints.

Initialize population with random bus schedules

Set population size, crossover rate, mutation rate and number of generations.

Evaluate fitness of each schedule:

Calculate bus arrival and departure times

Calculate total passenger waiting time.

Fitness = 1/(1+Total Waiting Time)

For each generation do

Select parent schedules based on fitness

Perform crossover on selected parents to generate new schedules

Apply mutation to change bus departure time.

Check scheduling constraints and correct invalid schedules

Evaluate fitness of new schedules.

Create a new population using offspring & the best schedules

Update the best solution found so far.

Display best bus schedule & minimum passenger waiting time.

```
import random

# User input
n = int(input("Enter number of buses: "))
m = int(input("Enter number of passengers: "))

passengers = list(map(int, input(
    "Enter passenger arrival times (minutes): "
).split()))

start = int(input("Enter start time (minutes): "))
end = int(input("Enter end time (minutes): "))

POP_SIZE = 10
GENERATIONS = 50
MUTATION_RATE = 0.1

# Fitness function
def fitness(schedule):
    waiting = 0

    for p in passengers:
        buses = [b for b in schedule if b >= p]

        if buses:
            waiting += min(buses) - p
        else:
            waiting += end - p

    return 1 / (1 + waiting)

# Initial population
population = [
    sorted(random.sample(
        range(start, end + 1), n
    ))
    for _ in range(POP_SIZE)
]

# Genetic Algorithm
for generation in range(GENERATIONS):

    population.sort(key=fitness, reverse=True)
    new_population = [population[0]]

    while len(new_population) < POP_SIZE:
        p1, p2 = random.sample(population[:5], 2)

        # Crossover
        child = sorted(random.sample(p1 + p2, n))

        # Mutation
        if random.random() < MUTATION_RATE:
            i = random.randrange(n)
            child[i] = random.randint(start, end)
            child = sorted(child)

        new_population.append(child)

    population = new_population

# Best solution
best = max(population, key=fitness)

print("\nBest Bus Departure Schedule:", best)
print("Minimum Passenger Waiting Time:",
      round(1 / fitness(best) - 1), "minutes")
```

```
Enter number of buses: 3
Enter number of passengers: 4
Enter passenger arrival times (minutes): 10 20 35 50
Enter start time (minutes): 0
Enter end time (minutes): 60

Best Bus Departure Schedule: [10, 23, 36]
Minimum Passenger Waiting Time: 14 minutes
```